Bitácora práctica
Bootstrap seguro de Azure Databricks Dev

Azure, Terraform, Unity Catalog, identidades y OIDC

Tutorial reproducible de la sesión de implementación

Proyecto:	`proyecto-databricks-cicd`
Entorno:	Dev
Región:	East US 2 (`eastus2`)
Fecha de la sesión:	9 de octubre de 2026
Presupuesto mensual:	USD 58 (aprox. S/200)

Los identificadores se muestran truncados y no se incluyen códigos de acceso, tokens ni datos personales. Git y Terraform continúan siendo la fuente de verdad.

Índice

1. Resultado de la jornada **1**
1.1. Arquitectura lograda . . . 1

2. Modelo mental: dos capas de Terraform **1**

3. Paso 1: suscripción y control de costos **2**
3.1. Activación de Pay-As-You-Go . . . 2
3.2. Presupuesto y alertas . . . 2

4. Paso 2: Azure CLI, tenant y MFA **2**
4.1. Primer inicio de sesión . . . 2
4.2. Dos identidades con propósitos distintos . . . 3

5. Paso 3: región y aprovisionamiento del workspace **3**
5.1. Variables de Azure . . . 3
5.2. Plan revisado . . . 3

6. Paso 4: acceso administrativo a Databricks **4**
6.1. Por qué falló la cuenta personal . . . 4
6.2. Alta en el workspace . . . 4

7. Paso 5: configuración Terraform de Databricks **5**
7.1. Datos locales y nomenclatura correcta . . . 5
7.2. Identidad CI/CD y OIDC . . . 6
7.3. Grupo y carpetas . . . 6

8. Paso 6: Unity Catalog y recuperación del primer apply **6**
8.1. Fallo observado . . . 6
8.2. Decisión adoptada . . . 6

9. Verificación de costos al cerrar la sesión **7**
9.1. Qué sí puede facturarse . . . 7
9.2. Lista de cierre seguro . . . 7

10. Seguridad pendiente inmediata **8**

11. Qué continúa en la próxima sesión **8**

12. Comandos de diagnóstico reutilizables **8**
12.1. Azure . . . 8
12.2. Terraform Azure . . . 9
12.3. Terraform Databricks . . . 9

13. Referencias oficiales **9**

Conclusión **9**

1. Resultado de la jornada

La sesión terminó con un entorno Dev funcional y administrado por Terraform. Azure contiene un Resource Group y un workspace Azure Databricks Premium; Databricks contiene la identidad CI/CD, la política OIDC para GitHub, el grupo de desarrolladores, las carpetas del proyecto y un esquema aislado de Unity Catalog. El último `terraform plan` de ambas capas informó `No changes`.

Estado final: infraestructura creada, reconciliada y sin cómputo activo

Componente	Resultado confirmado
Suscripción Azure	Pay-As-You-Go activa y presupuesto mensual de USD 58.
Azure Databricks	Workspace `dbw-databricks-cicd-dev` en `eastus2`.
Terraform Azure	Resource Group y workspace bajo estado local; plan estable.
Terraform Databricks	Service principal, OIDC, grupo, permisos, carpetas y esquema; plan estable.
Unity Catalog	Catálogo administrado del workspace y esquema `cicd`.
Costo al cierre	Clúster registrado detenido, SQL warehouse detenido y sin jobs activos.

1.1. Arquitectura lograda

```
GitHub (futuro workflow con OIDC)
       |
       v
sp-dbx-cicd-dev
       |
       +--> /Workspace/data/proyecto_test (codigo desplegado)
       +--> /Workspace/.cicd/proyecto_test/dev (estado protegido)
       +--> catalogo_del_workspace.cicd (pruebas del pipeline)

Azure subscription
       +--> rg-databricks-cicd-dev
               +--> dbw-databricks-cicd-dev
```

La automatización de GitHub todavía no fue activada. Lo realizado hoy prepara el destino y la identidad a la que el workflow se autenticará sin Personal Access Tokens.

2. Modelo mental: dos capas de Terraform

El laboratorio separa dos responsabilidades para que los cambios sean pequeños y comprensibles:

1. **Capa Azure** (`infra/azure/dev`): crea recursos de Azure, como el Resource Group y el workspace.
2. **Capa Databricks** (`infra/databricks/dev`): usa un workspace ya existente y crea objetos internos, identidades, permisos y gobierno de datos.

> **Concepto clave.** Un **provider** enseña a Terraform a hablar con una API. Un **resource** describe un objeto que Terraform administra. Una **variable** introduce un valor configurable, un **output** publica un resultado no sensible y el **state** vincula el código con los objetos reales. `plan` propone; `apply` modifica; `destroy` elimina.

El estado es local durante esta primera fase didáctica. Los archivos `terraform.tfstate`, `*.tfplan` y `terraform.tfvars` están excluidos de Git. Antes de incorporar QA se migrará el estado a un backend remoto con bloqueo y control de acceso.

3. Paso 1: suscripción y control de costos

3.1. Activación de Pay-As-You-Go

Se convirtió la cuenta inicial a Pay-As-You-Go y se creó la suscripción `Azure_Databricks_CICD_Lab`. La suscripción quedó en estado *Active*. La cuenta personal que creó la suscripción conserva el rol Owner y se usa para las operaciones de Azure.

El nombre de la suscripción sirve para reconocer el laboratorio; no cambia la seguridad ni el precio. El identificador real de la suscripción se trató como configuración local y en este documento se representa como `ba527c78-...-94dd`.

3.2. Presupuesto y alertas

Se creó manualmente el presupuesto `budget-proyecto-databricks-dev` con estas propiedades:

- importe: USD 58 por mes, equivalente aproximado al objetivo de S/200;
- reinicio mensual;
- alertas de costo real al 50 %, 75 %, 90 % y 100 %;
- destinatarios de correo configurados por el propietario.

> **Seguridad.** Un presupuesto **no detiene** recursos cuando se alcanza el límite. Envía alertas. Para una respuesta automática haría falta un Action Group y una automatización cuidadosamente diseñada. Por ahora, el control efectivo es apagar cómputo, revisar Cost Analysis y mantener recursos efímeros.

El presupuesto ya creado en el portal no se duplicó desde Terraform. La administración del budget mediante código quedó desactivada para esta fase.

4. Paso 2: Azure CLI, tenant y MFA

4.1. Primer inicio de sesión

El primer `az login` respondió con `AADSTS50076`: Azure exigía autenticación multifactor por un cambio de política o de ubicación. No era un error de contraseña; significaba que el flujo necesitaba interacción adicional.

La solución fue especificar el tenant y usar Device Code:

```
az login ‘
  --tenant "<TENANT_ID>" ‘
  --use-device-code

az account set --subscription "<SUBSCRIPTION_ID>"

az account show ‘
  --query "{name:name, id:id, state:state, tenantId:tenantId}" ‘
```

```
  --output table
```

La salida esperada mostró la suscripción del laboratorio en estado `Enabled` y el tenant correcto. Los códigos de Device Code son temporales y nunca deben copiarse a documentación o Git.

4.2. Dos identidades con propósitos distintos

Durante el bootstrap se usaron dos contextos:

- **Cuenta propietaria de Azure**: ve la suscripción y ejecuta Terraform sobre Azure.
- **Usuario organizacional de Entra**: entra al Account Console de Databricks y ejecuta el bootstrap de objetos a nivel de cuenta.

El usuario organizacional apareció como cuenta a nivel de tenant sin suscripciones. Eso es correcto: no necesita ser Owner de Azure para administrar temporalmente el Account Console de Databricks.

5. Paso 3: región y aprovisionamiento del workspace

Se revisaron las regiones disponibles y se eligió `eastus2`. La decisión priorizó disponibilidad del servicio y opciones de cómputo modernas. El costo exacto se evaluará con uso real, porque depende del tipo y duración del cómputo, no sólo de la región.

5.1. Variables de Azure

El archivo local `infra/azure/dev/terraform.tfvars` contiene nombres, región, tags y el ID de suscripción. No contiene contraseñas ni tokens y no debe versionarse.

```
subscription_id = "<SUBSCRIPTION_ID>"
location = "eastus2"
owner = "<OWNER_TAG>"
```

Los tags aplicados fueron `project`, `environment`, `owner` y `managed_by=terraform`. Permiten identificar costos y responsabilidad.

5.2. Plan revisado

Como Terraform no estaba instalado globalmente, se utilizó el binario descargado dentro de la carpeta de caché del proyecto:

```
& ".\.cache\terraform-1.13.4\terraform.exe" `
  -chdir="infra/azure/dev" `
  plan -input=false
```

El plan propuso exactamente dos recursos:

```
Plan: 2 to add, 0 to change, 0 to destroy.

+ azurerm_resource_group.this
+ azurerm_databricks_workspace.this
```

Se revisaron nombre, región, SKU Premium, Managed Resource Group y tags antes de autorizar el `apply`. Después del aprovisionamiento se confirmó el workspace en el portal y un nuevo plan informó:

```
No changes. Your infrastructure matches the configuration.
```

Concepto clave. Un plan estable demuestra **idempotencia**: ejecutar de nuevo la misma definición no propone duplicados. También detecta drift si alguien modifica manualmente un recurso administrado.

6. Paso 4: acceso administrativo a Databricks

6.1. Por qué falló la cuenta personal

El enlace del Account Console rechazó la cuenta personal de Microsoft con `AADSTS500200`. La aplicación esperaba una identidad organizacional de Microsoft Entra ID. Además, el primer intento de usar un correo sugerido falló porque ese usuario todavía no existía.

La corrección fue crear explícitamente un usuario organizacional llamado *Databricks Lab Admin* en el tenant y asignarle temporalmente el rol Global Administrator. Ese rol permitió completar el bootstrap inicial del Account Console.

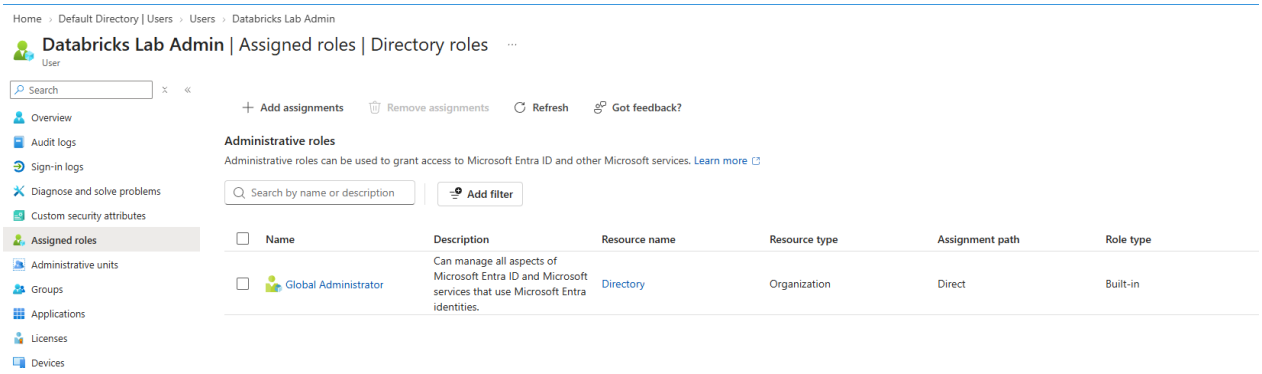


Figura 1: Rol temporal Global Administrator asignado al usuario organizacional del laboratorio.

Seguridad. Global Administrator es un rol extremadamente privilegiado. Ya no es necesario para el trabajo cotidiano del laboratorio. Debe retirarse al finalizar el bootstrap y conservarse únicamente el acceso específico requerido en Databricks.

6.2. Alta en el workspace

El usuario organizacional se agregó al workspace con los entitlements necesarios para administrarlo durante el bootstrap. Se mantuvieron desactivadas la creación irrestricta de clústeres y la creación de pools.

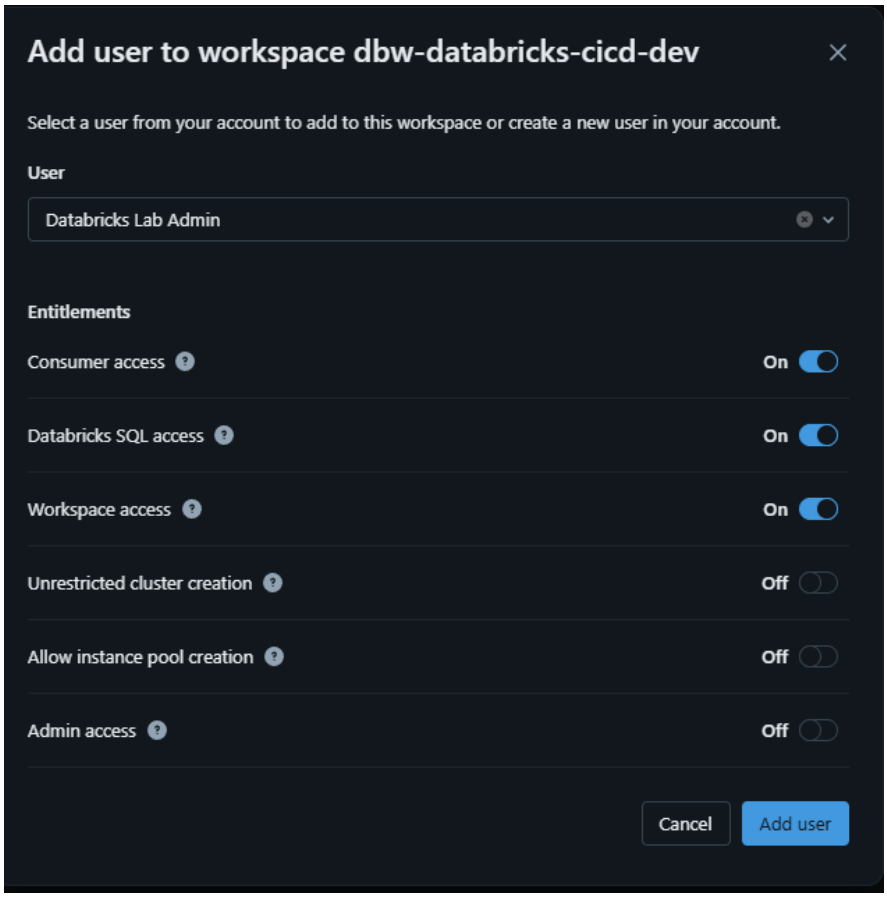


Figura 2: Entitlements usados para agregar el usuario administrativo al workspace.

Esta separación es importante: un rol de directorio de Entra no equivale a un entitlement o permiso dentro de Databricks.

7. Paso 5: configuración Terraform de Databricks

7.1. Datos locales y nomenclatura correcta

Se completó `infra/databricks/dev/terraform.tfvars` con:

- Account ID de Databricks, representado aquí como `96099a08-...-8274`;
- Workspace ID y host producidos por la capa Azure;
- propietario GitHub `dbernalesv`;
- nombre de repositorio `proyecto-databricks-cicd`;
- usuario de desarrollo y grupo `data-engineers`;
- catálogo administrado asociado al workspace.

La variable `github_repository` recibe sólo el nombre, no la URL completa. Terraform arma el subject OIDC con esta forma:

```
repo:<GITHUB_OWNER>/<GITHUB_REPOSITORY>:environment:dev
```

Por eso el valor correcto es `proyecto-databricks-cicd`, mientras que `github_owner` es `dbernalesv`.

7.2. Identidad CI/CD y OIDC

Terraform creó el service principal `sp-dbx-cicd-dev`. Su Application ID se tratará como variable de GitHub y se representa aquí como `89693b08-...-86fe`.

La política de federación confía únicamente en tokens emitidos por GitHub Actions para el repositorio y el Environment `dev` exactos. El audience es el Account ID de Databricks.

> **Concepto clave.** OIDC intercambia un token de corta duración por acceso durante el job. No hay un PAT permanente que guardar o rotar. Para funcionar de forma segura, GitHub debe conceder `id-token: write` sólo al job de despliegue y el Environment debe estar protegido.

7.3. Grupo y carpetas

Terraform administra el grupo de cuenta `data-engineers`, agrega al usuario de desarrollo y lo asigna al workspace. Los permisos resultantes son:

Objeto	Principal	Permiso
`/Workspace/data/proyecto_test`	`data-engineers`	`CAN_EDIT`
`/Workspace/data/proyecto_test`	Service principal CI/CD	`CAN_MANAGE`
`/Workspace/.cicd/proyecto_test/dev`	Service principal CI/CD	`CAN_MANAGE`

Dev permite edición manual para debugging. Sin embargo, el siguiente despliegue reconstruirá la carpeta desde el SHA de Git y eliminará archivos que no estén en el manifiesto. Los cambios manuales nunca se promueven a QA o Prod.

8. Paso 6: Unity Catalog y recuperación del primer apply

8.1. Fallo observado

El primer `terraform apply` avanzó lo suficiente para crear identidades, grupo y carpetas, pero falló al intentar crear un catálogo adicional. El workspace nuevo usaba *Default Storage* y no exponía una raíz de metastore válida para ese catálogo extra.

Este fallo no implicó borrar lo creado. Terraform registró en el state cada recurso confirmado y el siguiente plan calculó sólo lo pendiente.

8.2. Decisión adoptada

Se inspeccionó el workspace y se encontró el catálogo administrado creado automáticamente, respaldado por Default Storage. En vez de forzar almacenamiento adicional, la configuración se corrigió para:

1. leer el catálogo existente con un `data source`;
2. crear dentro de él el esquema `cicd`;
3. conceder al service principal `USE_CATALOG`;
4. conceder en el esquema `USE_SCHEMA`, `CREATE_TABLE`, `MODIFY` y `SELECT`.

La decisión está registrada en `docs/adr/0005-workspace-catalog-default-storage.md`. El apply de recuperación completó los objetos pendientes y el plan final volvió a ser estable:

```
No changes. Your infrastructure matches the configuration.
```

> **Concepto clave.** Este caso muestra por qué no se debe improvisar borrando state o recursos después de un fallo. Terraform es transaccional por recurso, no por ejecución completa: conserva lo que la API confirmó y permite reanudar después de corregir la causa.

9. Verificación de costos al cerrar la sesión

Antes de terminar se consultaron las APIs de Databricks con un token temporal obtenido desde Azure CLI. No se imprimió el token. El resumen fue:

```
Clusters registrados: 1
Clusters en ejecucion: 0
SQL warehouses registrados: 1
SQL warehouses en ejecucion: 0
Jobs activos: 0
```

El clúster y el SQL warehouse registrados están detenidos. No hay que eliminarlos esta noche para evitar gasto de cómputo. Tampoco es necesario destruir el workspace o el Resource Group: hacerlo eliminaría el avance y obligaría a repetir el bootstrap.

9.1. Qué sí puede facturarse

Azure Databricks factura principalmente DBUs y máquinas virtuales mientras el cómputo está activo. También pueden existir cargos pequeños por almacenamiento administrado, disco, red o direcciones públicas. Con el cómputo detenido, no se espera un consumo horario alto, pero el costo real debe confirmarse en Cost Analysis porque la facturación puede tener retraso.

9.2. Lista de cierre seguro

Al finalizar cada laboratorio:

1. abrir **Compute** y comprobar que todos los clústeres estén `TERMINATED`;
2. abrir **SQL Warehouses** y comprobar que estén `STOPPED`;
3. revisar **Jobs & Pipelines** y confirmar que no haya runs activos;
4. evitar crear endpoints de serving, pools o recursos de red permanentes sin presupuesto;
5. revisar **Cost Management >Cost Analysis** al día siguiente;
6. recordar que las alertas de budget no detienen el consumo.

Para clústeres interactivos futuros se configurará auto-termination. Para jobs se preferirá cómputo efímero o serverless, siempre con límites y timeouts.

10. Seguridad pendiente inmediata

La acción recomendada al retomar el proyecto es retirar el rol **Global Administrator** del usuario *Databricks Lab Admin*. El bootstrap ya concluyó. Este cambio no elimina el usuario, el workspace, el service principal ni la configuración Terraform; reduce el riesgo de una cuenta innecesariamente privilegiada.

También deben mantenerse estas reglas:

- no subir `terraform.tfvars`, state, planes, tokens ni archivos `.env`;
- no copiar tokens de Azure CLI a variables de GitHub;
- usar variables de GitHub Environment para IDs y hosts no secretos;
- usar OIDC para credenciales efímeras;
- no ejecutar workflows de PR no confiables con `id-token: write`.

11. Qué continúa en la próxima sesión

La infraestructura base ya no es el bloqueo. El siguiente tramo es conectar GitHub con Dev:

1. retirar el rol Global Administrator temporal;
2. crear o completar el GitHub Environment `dev`;
3. cargar como variables el host, Account ID y Client/Application ID, sin secretos persistentes;
4. proteger `main` con Pull Requests y checks obligatorios;
5. ejecutar `validate-pr.yml` y corregir cualquier validación;
6. probar el login OIDC desde GitHub Actions;
7. desplegar el Bundle a `/Workspace/data/proyecto_test`;
8. ejecutar el smoke test y confirmar el SHA desplegado;
9. revisar Cost Analysis después de la primera ejecución serverless.

QA y Prod permanecerán sin crear hasta que Dev despliegue de extremo a extremo de forma repetible.

12. Comandos de diagnóstico reutilizables

12.1. Azure

```
az account show ‘
  --query "{name:name, id:id, state:state, tenantId:tenantId}" ‘
  --output table

az resource list ‘
  --resource-group "rg-databricks-cicd-dev" ‘
  --output table
```

12.2. Terraform Azure

```
$tf = ".\.cache\terraform-1.13.4\terraform.exe"

& $tf -chdir="infra/azure/dev" fmt -check
& $tf -chdir="infra/azure/dev" validate
& $tf -chdir="infra/azure/dev" plan -input=false
```

12.3. Terraform Databricks

Antes de usar la capa Databricks hay que iniciar sesión con el usuario organizacional del tenant y obtener acceso a la aplicación Azure Databricks. Nunca se debe imprimir el token.

```
az login ‘
  --tenant "<TENANT_ID>" ‘
  --use-device-code ‘
  --allow-no-subscriptions

az account get-access-token ‘
  --resource "2ff814a6-3304-4ab8-85cb-cd0e6f879c1d" ‘
  --output none

& $tf -chdir="infra/databricks/dev" fmt -check
& $tf -chdir="infra/databricks/dev" validate
& $tf -chdir="infra/databricks/dev" plan -input=false
```

> **Seguridad.** No ejecutar `terraform destroy` como rutina de apagado. `destroy` elimina recursos y requiere revisar el objetivo exacto. Para controlar el gasto diario basta con detener cómputo. La destrucción del laboratorio se reservará para una decisión explícita y un plan revisado.

13. Referencias oficiales

- Microsoft Azure: precios de Azure Databricks.
- Microsoft Learn: crear y administrar presupuestos de Azure.
- Microsoft Learn: configuración y auto-termination de cómputo.
- Microsoft Learn: autenticación de Azure CLI.

Conclusión

La jornada no terminó únicamente con un workspace visible: terminó con infraestructura declarativa, identidad CI/CD sin secretos persistentes, permisos explícitos, un esquema aislado y evidencia de idempotencia. El entorno puede permanecer creado durante la noche porque su cómputo está detenido. La prioridad al regresar es reducir el privilegio del usuario de bootstrap y conectar GitHub Actions por OIDC para realizar el primer despliegue reproducible hacia Dev.